Hata Avcılığı: Yazılımda Kalite Pratikleri

ÖFY

October 2026

Özet

Yazılım kalitesi tek bir testle ya da tek bir araçla sağlanmaz. Her teknik bazı hata türlerini yakalar, bazılarını kaçırır. Bu yüzden kalite, art arda dizilmiş katmanlardan oluşan bir savunma sistemidir. İlk katman hatanın hiç yazılmamasını sağlar: küçük fonksiyonlar, düşük karmaşıklık, geçersiz durumu imkânsız kılan tipler. İkinci katman hatayı insan gözüyle yakalar: küçük ve tek amaçlı kod incelemeleri. Üçüncü katman hatayı testle yakalar. Ancak testlerin kendisi de yanılabilir; coverage oranı yüksek olan bir test paketi zayıf olabilir ve bunu en dürüst biçimde mutation testing gösterir. Dördüncü katman, insanın düşünemeyeceği girdileri makineye ürettirir: fuzzing, property-based testing ve sembolik çalıştırma. Beşinci katman canlı ortamdır; bazı hatalar yalnızca gerçek yük, gerçek veri ve gerçek ağ koşullarında ortaya çıkar. Son katman, bulunan her hatadan öğrenmektir: her bug bir regression testine, her ciddi olay bir postmortem'e dönüşür. Hataların büyük kısmı kodun küçük bir bölümünde toplandığı için test emeği eşit dağıtılmaz, riskli bölgelere yoğunlaştırılır.

Giriş

Kalite nedir

Yazılım kalitesi, bir yazılımın beklenen işi doğru, güvenilir ve sürdürülebilir biçimde yapma derecesidir. ISO/IEC 25010 standardı kaliteyi tek bir özellik olarak değil, bir özellikler kümesi olarak tanımlar: işlevsel uygunluk, performans, uyumluluk, kullanılabilirlik, güvenilirlik, güvenlik, bakım kolaylığı ve taşınabilirlik (ISO/IEC, 2011). Bu notta ağırlık, bu özelliklerin hepsini tehdit eden ortak düşmana verilmiştir: **bug**, yani yazılımın beklenenden farklı davranmasına yol açan kusur.

Test bir şeyi kanıtlamaz

Dijkstra, testin hatanın varlığını gösterebileceğini ama yokluğunu asla gösteremeyeceğini belirtmiştir (Dijkstra, 1970). Bir fonksiyon on bin girdiyle doğru çalışsa bile on bin birinci girdide çökebilir. Bu sınır, kalite çalışmasının temel mantığını belirler: amaç "hiç bug yok" demek değil, bug bulma olasılığını sistemli biçimde artırmaktır.

Katmanlı savunma

Reason'ın İsviçre peyniri modeli (*Swiss cheese model*), kazaları birbirinin önüne dizilmiş delikli peynir dilimleriyle açıklar (Reason, 1990). Her dilim bir savunma katmanıdır ve her katmanda delik vardır. Kaza, deliklerin tesadüfen aynı hizaya geldiği durumda olur. Yazılımda da durum aynıdır. Kod incelemesi mantık hatasını yakalar ama yarış durumunu (*race condition*) kaçırır. Race detector yarış durumunu yakalar ama yanlış iş kuralını kaçırır. Katmanların delikleri farklı yerlerde olduğu için birden çok katman birlikte kullanılır.

Yaygın bir yanılgı

Kalitenin test aşamasında "eklendiği" düşünülür. Oysa test, var olan kaliteyi ölçer; olmayan kaliteyi yaratamaz. Kötü tasarlanmış, birbirine sıkı bağlı bir kod tabanı test edilmesi zor bir kod tabanıdır ve bu yüzden az test edilir. Bu nedenle not, test tekniklerinden önce kodun yazılış biçimiyle başlar.

Kapsam

Not; kod yazımı, kod incelemesi, test tasarımı, otomatik hata arama, dağıtık sistemler, arayüz, güvenlik, canlı ortam ve hatadan öğrenme aşamalarındaki teknikleri kapsar. Örnekler çoğunlukla Go ve TypeScript/Angular ekosisteminden verilmiştir; tekniklerin kendisi dilden bağımsızdır. Kullanılabilirlik araştırması ve proje yönetimi kapsam dışıdır.

Birinci Katman: Hatayı Yazılmadan Önlemek

En ucuz bug hiç yazılmayan bug'dır. Bu katmandaki kurallar doğrudan bug bulmaz; bug'ın saklanabileceği yerleri azaltır.

Test-Driven Development

Test-Driven Development (TDD), kodu yazmadan önce testini yazma yöntemidir (Beck, 2002). Üç adımlık bir döngüyle çalışır:

1. **Kırmızı:** Henüz var olmayan davranış için başarısız bir test yazılır.
2. **Yeşil:** Testi geçirecek en az kod yazılır.
3. **Refactor:** Davranış korunarak kod temizlenir.

TDD'nin kaliteye katkısı iki yönlüdür. Birincisi, her davranışın baştan bir testi olur. İkincisi, test edilmesi zor kod yazmak zorlaşır; çünkü önce test yazıldığı için kod doğal olarak küçük ve bağımsız parçalara bölünür.

Boyut ve karmaşıklık sınırları

Aşağıdaki sınırlar katı kurallar değil, yaygın kabul görmüş sezgisel değerlerdir (*heuristic*). Kesin sayıların bilimsel bir dayanağı yoktur; işlevleri, kodun "fazla büyüdüğünü" erken fark ettirmektir.

Ölçü	Yaygın sınır
Fonksiyon uzunluğu	20–40 satır; bir ekrana sığmalı
Dosya uzunluğu	300–500 satırı geçerse bölünmeye aday
Cyclomatic complexity	Fonksiyon başına 10'un altı
Parametre sayısı	3–4'ü geçerse bir struct ile toplanır
İç içe girinti	En fazla 3 seviye

Cyclomatic complexity

Cyclomatic complexity, bir fonksiyondaki bağımsız çalışma yollarının sayısıdır (McCabe, 1976). Her `if`, `for`, `case` ve mantıksal `&&` yeni bir yol açar. Yol sayısı arttıkça her yolu test etmek zorlaşır ve test edilmeyen yollarda bug kalır. McCabe 10 değerini üst sınır olarak önermiştir. Go'da `gocyclo` ve `gocognit` araçları bu değeri ölçer.

Girintiyi azaltmak

Derin iç içe geçmiş `if` blokları, hangi koşulda hangi kodun çalıştığını izlemeyi zorlaştırır. Çözüm erken dönüştür (*guard clause*): hatalı durumlar fonksiyonun başında elenir, asıl iş girintisiz yazılır.

```
func Process(f *File) error {
    if f == nil {
        return ErrNilFile
    }
    if f.Size == 0 {
        return ErrEmptyFile
    }
    // asıl iş burada, girintisiz
    return save(f)
}
```

Tasarımla hatayı imkânsız kılmak

Geçersiz durumu temsil edilemez yapmak

Minsky'nin formüle ettiği ilkeye göre tipler, geçersiz bir durumun hiç oluşturulamayacağı şekilde tasarlanmalıdır (Minsky, 2011). Örneğin kullanıcı kimliği ile dosya kimliği ikisi de `string` olarak tutulursa, bir fonksiyona yanlışlıkla diğeri verilebilir ve derleyici bunu yakalamaz. Ayrı `UserID` ve `FileID` tipleri tanımlandığında aynı hata derleme aşamasında yakalanır. Bug test edilmeden önlenmiş olur.

Fail fast

Fail fast, hatalı bir durumla karşılaşıldığında sessizce devam etmek yerine hemen ve görünür biçimde hata vermektir. Sessizce devam eden program hatalı veriyi sistemin derinlerine taşır. Hata sonunda fark edildiğinde, kaynağı çok geride kalmıştır ve bulmak zorlaşır.

Assertion ve invariant

Invariant, bir programda her zaman doğru olması gereken koşuldur; örneğin "bakiye asla negatif olamaz". Bu koşullar koda assertion olarak yazıldığında bug, oluştuğu anda yakalanır. Meyer bu fikri *Design by Contract* adıyla sistemleştirmiştir: her fonksiyon için ön koşul, son koşul ve değişmez tanımlanır (Meyer, 1992).

Veritabanı kısıtları

`NOT NULL`, `UNIQUE`, `CHECK` ve foreign key kısıtları son savunma hattıdır. Uygulama kodu hatalı veri üretse bile veritabanı onu reddeder. Kod birden çok yerden veri yazıyorsa bu kısıtların değeri artar; çünkü kural tek bir yerde tutulur.

Strict tip kontrolü

TypeScript'te `strict` modu açıldığında `undefined` ve `null` kaynaklı hataların büyük kısmı derleme aşamasında yakalanır. Derleyici uyarılarının hata sayılması da aynı mantığa dayanır: görmezden gelinen uyarılar birikir ve gerçek uyarılar gürültüde kaybolur.

Kodu temiz tutma kuralları

- **Boy Scout kuralı:** Dokunulan kod, bulunduğundan biraz daha temiz bırakılır (Martin, 2008). Küçük iyileştirmeler zamanla birikir.
- **Üç kuralı** (*Rule of Three*): Aynı kod üçüncü kez yazıldığında soyutlanır, daha önce değil (Fowler, 1999). Erken soyutlama, yanlış soyutlamaya yol açar.
- **Ölü kod temizliği:** Kullanılmayan kod silinir. Bakımı yapılmayan kod yanlış anlaşılır ve yanlış yerde yeniden kullanılır. Go'da `deadcode` aracı bunu bulur.
- **Kopya kod tespiti:** Kopyalanıp küçük farkla değiştirilmiş kodlar bulunur. Bir kopyada düzeltilen bug, diğer kopyada genellikle kalır. Go'da `dupl` kullanılır.

Mimari kurallar

Mimari kurallar da testle zorunlu hale getirilebilir. "Handler katmanı veritabanına doğrudan erişemez" gibi bir kural yazıldığında, ihlal eden kod build'i kırar. Go'da `depguard` ve `go-arch-lint` bu iş için kullanılır. Paketler arasındaki döngüsel bağımlılıklar da aynı yolla engellenir.

İkinci Katman: İnsan Gözüyle Yakalamak

Kod incelemesi

PR boyutu

Cisco'da yapılan geniş bir kod incelemesi çalışması, bir seferde 200–400 satırdan fazla kod incelendiğinde hata bulma oranının belirgin biçimde düştüğünü göstermiştir (Cohen, 2006). İnceleyicinin dikkati sınırlıdır; 1000 satırlık bir PR'da yalnızca yüzeysel kontrol yapılır. Bu nedenle PR boyutu 200–400 satırla sınırlı tutulur.

Tek amaçlı PR

Bir PR tek bir iş yapar. Refactor ile yeni özellik aynı PR'a konursa, davranış değişikliği yapısal değişikliklerin arasında kaybolur ve fark edilmez.

İnceleme kontrol listesi

Her incelemede aynı sorular sorulur. Liste, inceleyicinin yorgun ya da aceleci olduğu anlarda bile temel kontrollerin atlanmamasını sağlar:

- Hata dönüşleri kontrol edilmiş mi?
- `nil` durumu düşünülmüş mü?
- Dosya, bağlantı gibi kaynaklar kapatılıyor mu?
- Ağ çağrılarında timeout var mı?
- Eşzamanlı erişilen veri korunuyor mu?

Fagan inspection

Fagan, rolleri belirlenmiş resmi bir inceleme yöntemi tanımlamıştır (Fagan, 1976). Toplantıda yazar, kodu okuyan kişi, moderatör ve kayıt tutan kişi bulunur. Yöntem ağırdır ama kritik modüllerde etkilidir.

Ek uygulamalar

- **İki onay kuralı:** Kritik modüllerde en az iki kişi onay verir.
- **Pair programming:** İki kişi aynı kodu birlikte yazar; hatalar yazılırken yakalanır.
- **LLM destekli inceleme:** Yapay zekâ araçları diff'i okuyup olası hataları işaretler. İnsan incelemesini tamamlar, yerini tutmaz.

Bireysel okuma teknikleri

- **Taze göz:** Kod bir gün bekletildikten sonra okunur. Yazarken akılda olan varsayımlar silindiği için, kodun gerçekte ne yaptığı görünür.
- **Tersten okuma:** Kod sondan başa, fonksiyon fonksiyon okunur. Akışa kapılmadan her parça tek başına değerlendirilir.
- **Rubber duck debugging:** Kod satır satır birine ya da bir nesneye anlatılır (Hunt ve Thomas, 1999). Anlatma zorunluluğu, atlanan varsayımları açığa çıkarır.

Gereksinim aşamasında kalite

Bug'ların önemli bir kısmı kodda değil, eksik ya da çelişkili gereksinimde başlar. Kod gereksinime birebir uysa bile yanlış iş yapar. Bu nedenle gereksinimler kod yazılmadan önce incelenir.

Behavior-Driven Development (BDD), gereksinimleri somut örneklerle yazma yöntemidir (North, 2006). Her davranış "Given–When–Then" kalıbıyla tanımlanır:

```
Given  kullanıcının sepetinde 3 ürün var
When   kullanıcı bir ürünü siler
Then   sepette 2 ürün kalır
```

Somut örnek yazmaya çalışırken belirsizlikler ortaya çıkar: "Ya sepette tek ürün varsa ve o silinirse?" Bu soru kod yazılmadan önce sorulmuş olur.

Threat modeling

Threat modeling, tasarım aşamasında "kim, nereden, neyi bozabilir" sorusunun sistemli biçimde sorulmasıdır. STRIDE yöntemi altı tehdit türünü kontrol listesi olarak kullanır: kimlik sahteciliği, veri değiştirme, inkâr, bilgi sızdırma, hizmet engelleme ve yetki yükseltme (Shostack, 2014).

FMEA

Failure Mode and Effects Analysis (FMEA), her bileşen için "nasıl bozulabilir, etkisi ne olur, ne kadar olası" sorularının bir tabloda yanıtlanmasıdır. Yöntem askeri mühendislikten gelir. Riskler daha kod yazılmadan sıralanır ve test emeği en riskli bileşenlere yönlendirilir.

Üçüncü Katman: Testi Doğru Tasarlamak

Test piramidi

Test piramidi, test türlerinin oranını tanımlayan modeldir (Cohn, 2009). Tabanda çok sayıda hızlı unit test, ortada daha az integration test, tepede en az sayıda E2E test bulunur. Oran bu şekildedir çünkü üst katmandaki testler yavaş, kırılgan ve hata kaynağını göstermekte zayıftır. Bir E2E test başarısız olduğunda sorunun hangi katmanda olduğu belli değildir; bir unit test başarısız olduğunda tam yeri bellidir.

İyi testin özellikleri

- **Arrange–Act–Assert yapısı:** Her test hazırlık, eylem ve doğrulama bölümlerinden oluşur ve tek bir şeyi test eder.
- **Hız:** Unit test paketi birkaç saniyede bitmelidir. Yavaş testler daha seyrek çalıştırılır ve hatalar daha geç fark edilir.
- **Bağımsızlık:** Testler birbirine bağımlı olmamalıdır. Go'da `go test -shuffle=on` testleri rastgele sırayla çalıştırarak gizli bağımlılıkları açığa çıkarır.
- **Kararlılık:** Ara sıra başarısız olan test (*flaky test*) hemen düzeltilir ya da karantinaya alınır. Flaky testlerin en yaygın nedenleri asenkron bekleme, eşzamanlılık ve test sırası bağımlılığıdır (Luo vd., 2014). Flaky test zamanla tüm test paketine güveni yok eder; ekip kırmızı sonucu görmezden gelmeye başlar ve gerçek hatalar da görmezden gelinir.

Test smell

Test smell, testin kendisindeki kötü tasarım işaretleridir: hiç assertion içermeyen test, `sleep` ile bekleyen test, birden çok davranışı aynı anda doğrulayan test. Bu testler geçer ama hiçbir şeyi güvenilir biçimde doğrulamaz.

Girdi seçme teknikleri

Denklik sınıfları

Denklik sınıfları (*equivalence partitioning*), girdilerin aynı davranışı tetikleyen gruplara ayrılmasıdır (Myers, 1979). Yaşı 18–65 arasında kabul eden bir fonksiyon için üç sınıf vardır: 18'den küçük, 18–65 arası ve 65'ten büyük. Her sınıftan bir örnek yeterlidir; çünkü aynı sınıftaki girdiler aynı kod yolundan geçer.

Sınır değer analizi

Hatalar çoğunlukla sınıfların sınırlarında çıkar (Myers, 1979). Yukarıdaki örnekte 17, 18, 65 ve 66 değerleri mutlaka denenir. Sebep basittir: `<` yerine `<=` yazmak, en yaygın programlama hatalarından biridir ve yalnızca sınırda görünür. Genel

olarak 0, 1, maksimum, maksimum+1, negatif sayı, boş string, `nil` ve boş liste denenir.

Karar tablosu

Birden çok koşulun birlikte etkilediği kurallarda, koşulların bütün kombinasyonları bir tabloya yazılır. Tablo doldurulurken hiç düşünülmemiş kombinasyonlar görünür hale gelir.

Üye mi	Kupon var mı	Sepet > 500	İndirim
Evet	Evet	Evet	?
Evet	Evet	Hayır	?
Evet	Hayır	Evet	?
...	...	...	...

Her soru işareti, gereksinimde cevabı olması gereken bir sorudur.

Durum geçiş testi

Sistemin durumları ve aralarındaki geçişler çizilir; örneğin `beklemede` → `yayında` → `durduruldu`. Her geçerli geçiş test edilir. Asıl bug'lar ise geçersiz geçişlerde bulunur: durdurulmuş bir yayını tekrar durdurmak, hiç başlamamış bir yayını bitirmek.

Pairwise testing

Bir sistemde on parametre ve her birinde dört değer varsa, bütün kombinasyonların sayısı 4^{10}, yani bir milyonu aşar. **Pairwise testing**, her parametre çiftinin en az bir kez birlikte test edilmesini sağlayan çok daha küçük bir test kümesi üretir (Cohen vd., 1997). Yöntem şu gözleme dayanır: yazılım hatalarının büyük kısmı bir ya da iki parametrenin etkileşiminden çıkar; üç ve daha fazla parametrenin birlikte tetiklediği hatalar giderek azalır (Kuhn vd., 2004).

Error guessing

Error guessing, deneyime dayanan hata tahminidir. Bilinen tuzaklar listesi denenir:

- Saat dilimi farkı ve yaz saati geçişi
- Gece yarısı, ay sonu, yıl sonu, artık yıl
- Türkçe "İ/ı" harflerinin büyük-küçük harf dönüşümü
- Emoji ve çok baytlı karakterler
- Çok uzun dosya adı, boşluk içeren dosya yolu
- Aynı isteğin aynı anda iki kez gelmesi

Doğru cevabı bilmeden test etmek

Bazı sistemlerde bir girdi için doğru çıktının ne olduğu bilinmez; örneğin karmaşık bir ses işleme algoritmasında. Buna **test oracle problemi** denir. Aşağıdaki teknikler bu problemi aşar.

Golden ve snapshot test

Bir kez doğruluğu elle kontrol edilen çıktı dosyaya kaydedilir. Sonraki her çalıştırmada çıktı bu dosyayla karşılaştırılır; fark varsa test başarısız olur. **Approval testing** aynı fikrin, değişikliğin bir insan tarafından onaylanmasını gerektiren biçimidir.

Characterization test

Feathers, anlaşılmayan eski kodu değiştirmeden önce mevcut davranışın olduğu gibi testle kaydedilmesini önerir (Feathers, 2004). Bu testler kodun doğru davrandığını iddia etmez, yalnızca bugünkü davranışı sabitler. Değişiklik sonrası davranış farkı hemen yakalanır.

Gerçekçi bağımlılıklarla test

- **Testcontainers:** Testler mock yerine Docker içinde çalışan gerçek PostgreSQL ve MinIO ile yapılır. Mock, gerçek sistemin davranışını taklit eder ama tam taklit etmez; aradaki fark bug saklar.
- **Contract testing:** Frontend ile backend arasındaki API sözleşmesi test edilir. Bir taraf alan adını değiştirirse test kırılır. `Pact` bu alanda yaygın araçtır.
- **Şema doğrulama:** Testte her API cevabı OpenAPI şemasına göre kontrol edilir.
- **Anonimleştirilmiş gerçek veri:** Sahte test verisi genellikle "temiz"dir. Gerçek verideki boş alanlar, beklenmeyen karakterler ve aşırı uzun değerler ancak gerçek veri şekliyle yakalanır.

Zamana bağlı davranışı test etmek

Kod saati doğrudan `time.Now()` ile okursa, "ay sonunda ne olur" sorusunu test etmek için ay sonunu beklemek gerekir. Çözüm, saatin koda dışarıdan verilmesidir (*fake clock*):

```
type Clock interface {
    Now() time.Time
}

// Testte saat istenen ana ayarlanır
```

```
clock := NewFakeClock(time.Date(2026, 12, 31, 23, 59, 59, 0, time.UTC))
svc := NewService(clock)
clock.Advance(2 * time.Second) // yıl değişti
```

Özel durum testleri

- **Idempotency testi:** Aynı istek iki kez gönderilir. İki kayıt oluşuyor ya da işlem iki kez yapılıyorsa bug vardır. Ağ hataları nedeniyle istemcilerin isteği tekrarlaması normal bir durumdur.
- **Migration testi:** Her veritabanı migration'ı ileri ve geri çalıştırılır. Boş tabloda değil, dolu tabloda denenir; çünkü sorunlar genellikle mevcut veride çıkar.
- **Rollback testi:** Yeni sürüme geçip eski sürüme dönmenin gerçekten çalıştığı test edilir. Test edilmemiş geri dönüş planı, acil durumda çalışmayabilir.
- **Yedekten geri yükleme testi:** Yedekler düzenli olarak gerçekten geri yüklenir. Yedeğin bozuk olduğu çoğu zaman yalnızca bu şekilde anlaşılır.

Model-based testing

Model-based testing, sistemin basitleştirilmiş bir modelinin yazılması ve test senaryolarının bu modelden otomatik üretilmesidir (Utting ve Legeard, 2007). Durum geçiş diyagramı bir model örneğidir; araç, diyagramdaki bütün yolları kapsayan test dizilerini kendisi üretir.

Dördüncü Katman: Hatayı Makineye Aratmak

İnsan eliyle yazılan testler, yazarın aklına gelen durumlarla sınırlıdır. Bu katmandaki teknikler, insanın düşünmediği girdileri makineye ürettirir.

Fuzzing

Fuzzing, programa çok sayıda rastgele ya da bozuk girdi verip çökmesini, donmasını ya da beklenmeyen davranmasını arama yöntemidir. Miller ve arkadaşları 1990'da UNIX araçlarına rastgele girdi vererek araçların önemli bir kısmının çöktüğünü göstermiştir (Miller vd., 1990).

Fuzzing türleri

- **Coverage-guided fuzzing:** Fuzzer, kodda yeni bir dal açan girdileri saklar ve onları değiştirerek yeni girdiler üretir. Kör rastgele denemeden çok daha etkilidir. AFL ve libFuzzer bu yaklaşımın bilinen araçlarıdır. Go'nun yerleşik fuzzing desteği de bu türdendir.
- **Grammar-based fuzzing:** Girdi bir dilbilgisine uygun üretilir; örneğin geçerli JSON ya da SQL. Rastgele girdi ayrıştırıcıda hemen reddedilir; dilbilgisine uygun girdi ise ayrıştırıcıyı geçip daha derindeki kodu dener.
- **Stateful fuzzing:** Tek istek yerine istek dizileri üretilir: oluştur, sil, sonra güncelle. Sıra bağımlı hatalar bu şekilde bulunur.
- **API fuzzing:** OpenAPI şemasından binlerce istek üretilip 500 hatası veren uçlar aranır. `Schemathesis` ve `RESTler` bu iş için kullanılır.

Go'da bir fuzz testi şöyle yazılır:

```
func FuzzParseConfig(f *testing.F) {
    f.Add([]byte(‘{"name":"test"}‘)) // başlangıç örneği
    f.Fuzz(func(t *testing.T, data []byte) {
        cfg, err := ParseConfig(data)
        if err != nil {
            return // hata dönmesi sorun değil
        }
        // çökmemeli ve tekrar serileştirilebilmeli
        if _, err := json.Marshal(cfg); err != nil {
            t.Fatalf("geçerli config serileştirilemedi: %v", err)
        }
    })
}
```

`go test -fuzz=FuzzParseConfig` komutuyla çalıştırılır. Google'ın OSS-Fuzz hizmeti, açık kaynak projeler için sürekli çalışan ücretsiz fuzzing altyapısı sağlar.

Property-based testing

Property-based testing, tek tek örnekler yerine her girdi için doğru olması gereken kuralların (*property*) yazılmasıdır. Araç binlerce rastgele girdi üretir ve kuralı bozan bir örnek bulursa, onu en küçük hâline indirerek raporlar. Yöntem Haskell için geliştirilen QuickCheck ile yaygınlaşmıştır (Claessen ve Hughes, 2000).

Tipik kurallar şunlardır:

- **Gidiş-dönüş:** Encode edilip decode edilen veri orijinaliyle aynı olmalıdır.
- **Değişmezlik:** Sıralanan listenin uzunluğu değişmemelidir.
- **Idempotency:** Bir listeyi iki kez sıralamak, bir kez sıralamakla aynı sonucu vermelidir.

Go'da `rapid`, TypeScript'te `fast-check` kullanılır.

Karşılaştırmalı teknikler

Differential testing

Differential testing, aynı girdinin aynı işi yapan iki farklı uygulamaya verilip sonuçların karşılaştırılmasıdır (McKeeman, 1998). Eski ve yeni sürüm, kendi yazılan kod ile hazır bir kütüphane ya da iki farklı derleyici karşılaştırılabilir. Sonuçlar farklıysa en az birinde bug vardır.

Metamorphic testing

Metamorphic testing, doğru cevabın bilinmediği durumlarda girdiler arasındaki ilişkilerin test edilmesidir (Chen vd., 1998). Örneğin bir ses dosyasının seviyesi 2 kat yükseltilip sonra 2 kat düşürüldüğünde orijinale yakın sonuç çıkmalıdır. Bir arama motorunda filtre eklemek sonuç sayısını artıramaz. Tek tek çıktıların doğruluğu bilinmese bile bu ilişkilerin bozulması hatayı gösterir.

Shadow traffic

Canlı ortamdaki gerçek istekler yeni sürüme de kopyalanır, iki sürümün cevapları karşılaştırılır. Kullanıcı yalnızca eski sürümün cevabını görür. Bu yöntem, differential testing'in canlı veriyle yapılan biçimidir.

Testlerin gücünü ölçmek: mutation testing

Mutation testing, koda bilerek küçük hatalar ekleyip testlerin bunları yakalayıp yakalamadığını ölçer (DeMillo vd., 1978). Örneğin `>` işareti `>=` yapılır, bir `return` değeri değiştirilir. Bu değiştirilmiş koda *mutant* denir. Testler başarısız olursa mutant "öldürülmüştür"; testler hâlâ geçiyorsa mutant "hayatta kalmıştır" ve o bölgede gerçek bir bug fark edilmeden kalabilir.

Mutation testing, coverage'dan daha dürüst bir ölçüdür. Coverage yalnızca bir satırın çalıştırıldığını gösterir, sonucunun doğrulandığını göstermez. Hiç assertion içermeyen bir test bile coverage'ı %100 yapabilir; ama hiçbir mutantı öldüremez. Go için `gremlins`, TypeScript için `Stryker` kullanılır.

Sembolik çalıştırma

Symbolic execution, programın somut değerler yerine sembollerle çalıştırılmasıdır (King, 1976). Program `x` için 5 gibi bir değer almaz, "herhangi bir `x`" alır. Her `if` ifadesinde araç iki yolu da izler ve her yolu açmak için gereken koşulu toplar. Sonunda her yolu tetikleyen somut girdiyi bir çözücüyle hesaplar.

Concolic testing, sembolik çalıştırmanın gerçek çalıştırmayla birlikte kullanıldığı biçimidir (Godefroid vd., 2005). Tamamen sembolik çalıştırmada yol sayısı patladığı için bu karma yöntem pratikte daha kullanışlıdır. **Bounded model checking** ise bütün olası çalışmaları belirli bir derinliğe kadar matematiksel olarak dener; C kodu için CBMC bilinen araçtır.

Statik analiz

Statik analiz, kodu çalıştırmadan inceleyerek hata arama yöntemidir.

- **Go:** `go vet`, `staticcheck` ve `golangci-lint` içindeki `errcheck` (yutulan hatalar), `nilaway` (`nil` pointer) ve `ineffassign` (etkisiz atamalar).
- **TypeScript/Angular:** `eslint` ve TypeScript `strict` modu.
- **Taint analysis:** Kullanıcıdan gelen verinin temizlenmeden tehlikeli bir noktaya (SQL sorgusu, dosya yolu, sistem komutu) ulaşıp ulaşmadığı izlenir.
- **SAST:** Static Application Security Testing; kodun güvenlik açıkları için taranmasıdır. Go için `gosec`, genel kullanım için `semgrep`.

Çalışma zamanı dedektörleri

- **Race detector:** `go test -race`, ortak veriye kilitsiz eşzamanlı erişimi çalışma sırasında yakalar. WebSocket gibi çok sayıda goroutine çalıştıran kodda özellikle değerlidir.
- **Tekrarlı çalıştırma:** `go test -count=100 -race` testleri yüz kez çalıştırır. Nadir zamanlamalarda oluşan hatalar ortaya çıkar.
- **Goroutine sızıntısı:** `goleak` paketi, test bittiğinde kapanmamış goroutine kalıp kalmadığını kontrol eder.
- **Sanitizer'lar:** C/C++ ve cgo kodunda bellek taşması (AddressSanitizer), başlatılmamış bellek (MemorySanitizer) ve tanımsız davranış (UndefinedBehaviorSanitizer) çalışırken yakalanır. Valgrind aynı işi daha yavaş yapar.

Beşinci Katman: Sistem ve Dayanıklılık

Tek tek doğru çalışan bileşenler, bir araya geldiğinde hatalı davranabilir. Bu katman bileşenlerin etkileşimini ve kötü koşullara dayanıklılığını test eder.

Hata enjeksiyonu

Chaos testing

Chaos testing, sisteme bilerek arıza verilip davranışının gözlenmesidir (Rosenthal ve Jones, 2020). Ağ bağlantısı kesilir, veritabanı yavaşlatılır, depolama servisi hata döndürür. Amaç, arızanın gerçek hayatta olmasını beklemeden sistemin nasıl tepki verdiğini görmektir. `Toxiproxy` ağ arızalarını taklit eden bir araçtır.

Failpoint

Failpoint, kodun belirli noktalarına testte tetiklenebilen hata noktaları konmasıdır. Örneğin "bu satırda disk dolu hatası ver" denir. Böylece gerçekte nadiren oluşan hata yolları düzenli olarak test edilir. Go'da `gofail` kullanılır.

Dağıtık sistem hataları

- **Ağ bölünmesi** (*network partition*): Sunucular arasındaki bağlantı kesilir ve veri tutarlılığı kontrol edilir. Kingsbury'nin Jepsen çalışmaları, birçok yaygın veritabanında bu yolla tutarlılık hataları bulmuştur.
- **Saat kayması:** Sunucuların saatleri bilerek birbirinden farklı ayarlanır. Zamana dayalı sıralama yapan sistemler bu durumda hatalı sonuç üretebilir.
- **Mesaj sırası ve tekrarı:** Mesajlar ters sırada, gecikmeli ya da iki kez gönderilir.
- **Retry storm:** Bir servis yavaşladığında bütün istemciler aynı anda tekrar dener ve servisi tamamen çökertir. Exponential backoff ve jitter bunu önler; test, bu korumanın gerçekten çalıştığını doğrular.

Yük altında davranış

Test türü	Ne bulur
Yük testi	Beklenen yükte yavaşlama ve hata
Spike test	Trafik aniden 10 katına çıktığında çökme ve toparlanma
Soak test	Saatlerce ya da günlerce çalışmada biriken bellek sızıntısı, dolan disk, tükenen bağlantı havuzu

Go için `vegeta`, genel kullanım için `k6` yaygın araçlardır.

Deterministic simulation testing

Dağıtık sistemlerdeki nadir hatalar, ağ gecikmesi ve iş parçacığı zamanlaması gibi kontrol edilemeyen etkenlere bağlıdır. Bu yüzden bir kez görülen hata tekrar üretilemez. **Deterministic simulation testing**, ağı, zamanı ve rastgeleliği tek bir kontrollü simülasyonun içine alır. Aynı başlangıç değeri (*seed*) her zaman aynı çalışmayı üretir; böylece bulunan her hata tekrar oynatılabilir. FoundationDB bu yaklaşımın bilinen uygulamasıdır (Zhou vd., 2021).

Formal yöntemler

- **TLA+:** Lamport'un geliştirdiği bu dil ile sistemin tasarımı matematiksel olarak tanımlanır ve bütün olası durumlar otomatik kontrol edilir (Lamport, 2002). Eşzamanlılık ve kilitlenme hataları daha kod yazılmadan bulunur.
- **Proof assistant:** Coq, Lean ve Dafny gibi araçlarla kodun doğruluğu matematiksel olarak ispatlanır. Maliyeti yüksektir; kritik algoritmalarda kullanılır.

N-version programming

N-version programming, aynı şartnamenin bağımsız takımlarca ayrı ayrı kodlanması ve sonuçların karşılaştırılmasıdır (Avizienis, 1985). Havacılık gibi kritik alanlarda kullanılır. Ancak Knight ve Leveson, bağımsız takımların beklenenden sık olarak aynı hataları yaptığını göstermiştir (Knight ve Leveson, 1986). Bu nedenle yöntemin sağladığı güvence, bağımsızlık varsayımının gerçekte tam sağlanmaması ölçüsünde azalır.

Arayüz Katmanında Kalite

Arayüz bug'ları çoğunlukla mantık hatası değil, görsel ya da ortama bağlı hatalardır. Bu yüzden ayrı teknikler gerektirir.

- **Visual regression:** Sayfaların ekran görüntüsü alınır ve önceki sürümle piksel düzeyinde karşılaştırılır. Kaymış bir buton ya da bozulmuş bir düzen, işlevsel testlerin göremeyeceği bir hatadır. Playwright ve Percy bu iş için kullanılır.
- **Cross-browser test:** Uygulama Chrome, Firefox ve Safari'de ayrı ayrı denenir. Tarayıcılar aynı standardı farklı biçimlerde uygulayabilir.
- **Pseudo-localization:** Bütün metinler yapay olarak uzatılır ve aksanlı harflere çevrilir. Taşan yazılar ve çevrilmeden kalmış sabit metinler hemen görünür.
- **Erişilebilirlik taraması:** `axe` aracı eksik etiketleri, yetersiz renk kontrastını ve klavyeyle ulaşılamayan öğeleri bulur.
- **Heap snapshot karşılaştırması:** Chrome DevTools'ta bir sayfaya girilip çıkılır ve bellek iki kez ölçülür. Bellek artmışsa sızıntı vardır. Angular uygulamalarında en yaygın sebep kapatılmayan abonelik ya da event listener'dır.
- **Monkey testing:** Arayüze rastgele tıklama, yazma ve kaydırma uygulanır. `gremlins.js` bu iş için kullanılır. Fuzzing'in arayüz karşılığıdır.
- **Ağ kısıtlama:** DevTools'ta yavaş bağlantı ya da çevrimdışı mod seçilir. Yükleniyor durumları, zaman aşımları ve hata ekranları kontrol edilir.
- **Lighthouse:** Performans, erişilebilirlik ve en iyi uygulama hatalarını raporlar.
- **Bundle boyutu takibi:** Build çıktısının boyutu her PR'da izlenir. Ani büyüme, yanlışlıkla eklenmiş büyük bir bağımlılığa işaret eder.

Güvenlik Kalitesi

Güvenlik açığı, kötü niyetli birinin bilerek tetiklediği bir bug türüdür. Normal kullanımda hiç ortaya çıkmayabilir; bu yüzden ayrıca aranır.

- **SAST:** Kod çalıştırılmadan güvenlik açığı için taranır (önceki bölümde anlatıldı).
- **DAST:** Dynamic Application Security Testing; çalışan uygulamaya dışarıdan saldırı denenir. `OWASP ZAP` bu alanda yaygın açık kaynak araçtır.
- **Bağımlılık taraması:** Kullanılan kütüphanelerdeki bilinen açıklar bulunur. Go için `govulncheck`, npm için `npm audit` kullanılır.
- **İmaj taraması:** Docker imajlarındaki açıklı sistem paketleri bulunur. `trivy` bu iş için kullanılır.
- **Penetrasyon testi:** Bir uzman sisteme gerçekten sızmaya çalışır. Araçların bulamayacağı mantık açıklarını bulur.
- **Bug bounty:** Güvenlik açığı bulan dış araştırmacılara ödül verilir. Böylece çok sayıda farklı bakış açısı sisteme yönlendirilir.

Altıncı Katman: Teslim ve Canlı Ortam

Sürekli entegrasyonda kalite kapıları

Quality gate, bir değişikliğin ana dala alınabilmesi için geçmesi gereken otomatik koşullar kümesidir (Duvall vd., 2007). Tipik kapılar şunlardır:

- Linter ve formatter hatasız geçmeli.
- Bütün testler geçmeli, race detector temiz olmalı.
- Yeni kodun coverage oranı mevcut oranı düşürmemeli.
- Karmaşıklık ve kod tekrarı belirlenen sınırı aşmamalı.
- Bağımlılık taramasında kritik açık olmamalı.
- Benchmark sonuçları anlamlı bir yavaşlama göstermemeli.

Kapıların değeri, insan unutkanlığından bağımsız olmalarıdır. Kural yazılıysa ama zorunlu değilse, zaman baskısı altında atlanır.

Ortam ve yapılandırma

- **Config linting:** Dockerfile (`hadolint`), Kubernetes dosyaları (`kubeconform`) ve Terraform (`tflint`) hata için taranır. Yapılandırma hataları kod hataları kadar sistemi bozabilir.
- **Ortam eşitliği:** Test, staging ve canlı ortamların yazılım sürümleri ve ayarları karşılaştırılır. "Bende çalışıyor" türü hataların kaynağı genellikle ortam farkıdır.

Kontrollü yayın

Humble ve Farley, yayın riskini azaltmak için değişikliğin kademeli olarak açılmasını önerir (Humble ve Farley, 2010).

- **Feature flag:** Yarım kalmış iş ana dalda kapalı bir bayrak arkasında durur. Uzun ömürlü branch'lerin birleştirilmesindeki hatalar önlenir.
- **Canary deploy:** Yeni sürüm önce kullanıcıların küçük bir kısmına açılır. Hata oranı artarsa geri alınır; hatadan etkilenen kullanıcı sayısı sınırlı kalır.
- **Dark launch:** Yeni kod canlıda çalıştırılır ama sonucu kullanıcıya gösterilmez. Yalnızca davranışı ve performansı izlenir.

Canlı ortamda izleme

Hiçbir test ortamı canlı ortamı tam olarak taklit edemez. Bu nedenle bazı bug'lar ancak canlıda bulunur; amaç onları kullanıcıdan önce fark etmektir.

- **Hata takibi:** Sentry gibi araçlar canlıdaki hataları stack trace ve bağlam bilgisiyle toplar.
- **Yapısal loglama ve tracing:** Bir isteğin bütün servislerdeki yolu tek bir kimlikle izlenir. OpenTelemetry bu alanda standarttır.
- **Synthetic monitoring:** Bir bot, belirli aralıklarla canlı sistemde gerçek kullanıcı gibi işlem yapar ve başarısızlığı raporlar.
- **SLO ve hata bütçesi: Service Level Objective (SLO)**, "isteklerin %99,9'u başarılı olmalı" gibi ölçülebilir bir hedeftir. Hedefle gerçek arasındaki pay hata bütçesidir; bütçe tükenirse yeni özellik yerine güvenilirlik çalışmasına öncelik verilir (Beyer vd., 2016).
- **Anomali tespiti:** Log ve metriklerde normalden sapan davranış otomatik işaretlenir.
- **Session replay:** Hata alan kullanıcının ekranda ne yaptığı kaydedilip izlenir. Kullanıcının tarif edemediği hata adımları görünür.

Veri bütünlüğü

- **Checksum kontrolü:** Dosyalar yüklendikten, kopyalandıktan ya da taşındıktan sonra hash değerleri karşılaştırılır. Sessizce bozulan veri bu şekilde yakalanır.
- **Mutabakat** (*reconciliation*): İki kaynaktaki veri düzenli olarak karşılaştırılır; örneğin veritabanındaki kayıtlar ile nesne depolamadaki dosyalar. Sahipsiz dosyalar ve dosyası olmayan kayıtlar bulunur.

Kullanıcıyla test

- **Dogfooding:** Takım kendi ürününü günlük işinde kullanır. Gerçek kullanım, senaryoya bağlı testlerin kaçırdığı hataları bulur.
- **Beta testi:** Ürün sınırlı sayıda gerçek kullanıcıya erken açılır.
- **Exploratory testing:** Belirli bir süre boyunca uygulama bilinçli olarak bozulmaya çalışılır: hızlı tıklama, geri tuşu, iki sekme, ağı kesme. Senaryo yoktur; test eden kişinin merakı yön verir.
- **Bug bash:** Takım belirli bir süre boyunca toplu halde exploratory testing yapar.

Yedinci Katman: Bulunan Hatadan Öğrenmek

Bir bug'ı düzeltmek, onu bir daha olmayacak hale getirmekle aynı şey değildir. Bu katman, bulunan her hatanın kalıcı bir korumaya dönüşmesini sağlar.

Hatanın kaynağını bulmak

- `git bisect`**:** Bug'ı getiren commit ikili aramayla bulunur. 1000 commit arasında yaklaşık 10 adımda sonuca ulaşılır; çünkü her adım aday sayısını yarıya indirir.
- **Delta debugging:** Hataya yol açan büyük bir girdi sistematik olarak küçültülür ve hatayı tetikleyen en küçük parça bulunur (Zeller ve Hildebrandt, 2002).
- **Debugger:** Go'da `delve` ile kod adım adım ilerletilir ve değişkenler izlenir.
- **Time-travel debugging:** Program çalışırken kaydedilir ve geriye doğru oynatılabilir. Hatanın oluştuğu ana geri dönülür. `rr` bu alanda bilinen araçtır.
- **Core dump analizi:** Çöken programın o anki bellek görüntüsü incelenir.
- **Profiling:** Go'da `pprof` ile bellek sızıntısı ve aşırı CPU kullanımının kaynağı bulunur.
- `EXPLAIN ANALYZE`**:** PostgreSQL'de yavaş sorgunun neden yavaş olduğunu gösterir; örneğin eksik index. Döngü içinde atılan sorgular (*N+1 sorgu problemi*) loglardan sayılarak bulunur.

Regression testi

Her bug için önce onu yeniden üreten bir test yazılır, sonra bug düzeltilir. Test önce başarısız olur, düzeltmeden sonra geçer. Bu sıra iki şeyi kanıtlar: testin gerçekten bu bug'ı yakaladığını ve düzeltmenin gerçekten işe yaradığını. Test kalıcı olarak kalır ve aynı bug bir daha geri gelemez.

Postmortem

Postmortem, ciddi bir olaydan sonra yazılan ve suçlu aramayan bir analiz belgesidir (Beyer vd., 2016). Suçlu aranmamasının sebebi, insanların hatalarını saklamaya başlamasını önlemektir. "5 Neden" yöntemiyle yüzeydeki sebepten kök sebebe inilir:

```
Neden kesinti oldu?      → Disk doldu.
Neden disk doldu?        → Loglar silinmedi.
Neden loglar silinmedi?  → Log rotasyonu ayarlı değildi.
Neden ayarlı değildi?    → Kurulum şablonunda yoktu.
Neden şablonda yoktu?    → Şablon hiç gözden geçirilmedi.
```

Kök sebep düzeltildiğinde yalnızca bu olay değil, aynı kaynaktan doğacak bütün benzer olaylar önlenir.

Game day

Game day, planlı bir arıza tatbikatıdır. Canlıya benzeyen bir ortamda bir bileşen bilerek bozulur ve ekibin tespit, müdahale ve geri dönüş süreci test edilir. Yalnızca yazılımın değil, insanların ve süreçlerin kalitesi de ölçülür.

Hataların dağılımı

Fenton ve Ohlsson, büyük bir telekomünikasyon sisteminde hataların büyük kısmının modüllerin küçük bir kısmında toplandığını göstermiştir (Fenton ve Ohlsson, 2000). Bu Pareto benzeri dağılımın pratik sonucu şudur: çok bug çıkan bir modülde büyük olasılıkla daha fazlası vardır ve test emeği oraya yoğunlaştırılmalıdır.

Hotspot analizi

Tornhill, git geçmişini kullanarak riskli dosyaları bulma yöntemi önermiştir (Tornhill, 2015). En sık değiştirilen ve aynı zamanda en karmaşık olan dosyalar "hotspot"tur. Sık değişim, dosyanın sürekli dokunulduğunu; karmaşıklık, her dokunuşta hata yapma olasılığının yüksek olduğunu gösterir. İnceleme ve refactor bu dosyalardan başlar.

Defect prediction

Geçmiş bug kayıtları, değişim sıklığı ve karmaşıklık ölçülerinden yola çıkarak hangi dosyalarda bug olma olasılığının yüksek olduğu istatistiksel olarak tahmin edilir. Fenton ve Neil, bu modellerin çoğunun nedensellik yerine yalnızca korelasyona dayandığını ve farklı projelere aktarıldığında başarısının düştüğünü eleştirmiştir (Fenton ve Neil, 1999). Bu nedenle tahmin modelleri kesin karar aracı olarak değil, dikkat yönlendirme aracı olarak kullanılır.

Kaliteyi Ölçmek

Takım düzeyinde: DORA metrikleri

Forsgren, Humble ve Kim, yazılım teslim performansını ölçen dört metrik tanımlamıştır (Forsgren vd., 2018):

Metrik	Neyi ölçer
Deploy sıklığı	Ne kadar sık canlıya çıkılıyor
Değişiklik süresi	Commit'ten canlıya geçen süre
Başarısız değişiklik oranı	Canlıya çıkan değişikliklerin yüzde kaçı soruna yol açıyor
Hatadan dönüş süresi (MTTR)	Bir arızadan sonra hizmetin ne kadar sürede düzeldiği

Çalışmanın önemli bulgusu, hız ile kalitenin birbirinin karşıtı olmadığıdır. Yüksek performanslı ekipler hem daha sık deploy etmekte hem de daha az başarısız değişiklik üretmektedir. Bunun nedeni, küçük ve sık değişikliklerin hem daha kolay test edilmesi hem de bozulduğunda daha kolay geri alınmasıdır.

Kod düzeyinde: coverage ve sınırları

Code coverage, testler çalışırken kodun yüzde kaçının çalıştırıldığını gösterir. Google'daki uygulamada %60 kabul edilebilir, %75 iyi, %90 örnek düzey olarak kabul edilmiştir (Ivanković vd., 2019).

Ancak coverage ile testin hata bulma gücü arasındaki ilişki zayıftır. Inozemtseva ve Holmes, test paketinin büyüklüğü sabit tutulduğunda coverage ile hata yakalama başarısı arasında güçlü bir ilişki bulunmadığını göstermiştir (Inozemtseva ve Holmes, 2014). Coverage bu nedenle "hangi kod hiç test edilmemiş" sorusuna yanıt olarak değerlidir; "testler iyi mi" sorusuna yanıt olarak değil. İkinci soru için mutation skoru daha güvenilir bir ölçüdür.

Pratik bir kural olarak sabit bir coverage hedefi yerine şu koşul uygulanır: yeni eklenen kod, mevcut coverage oranını düşürmemelidir.

Performans gerilemesi

- **Benchmark karşılaştırması:** Her PR'da benchmark çalıştırılır ve önceki sonuçla istatistiksel olarak karşılaştırılır. Go'da `benchstat` bu karşılaştırmayı yapar ve farkın gürültü mü yoksa gerçek mi olduğunu belirtir.
- **Allocation sayımı:** Go'da `testing.AllocsPerRun` ile bir fonksiyonun yaptığı bellek ayırma sayısı test edilir. Sayı artarsa test başarısız olur.

Test süresini kısa tutmak

Test impact analysis, değişen koda göre yalnızca ilgili testlerin seçilip çalıştırılmasıdır. Test paketi büyüdükçe bütün testleri her değişiklikte çalıştırmak yavaşlar. Hızlı geri bildirim, testlerin daha sık çalıştırılmasını sağlar.

Ölçünün tuzağı

Bir ölçü hedef haline geldiğinde, iyi bir ölçü olmaktan çıkar. Coverage hedefi konan bir ekip, assertion içermeyen ama coverage'ı artıran testler yazmaya başlayabilir. Satır sınırı konan bir ekip, bir fonksiyonu anlamsız parçalara bölebilir. Bu nedenle ölçüler tek başına değil, birbirini dengeleyecek biçimde birlikte kullanılır: coverage mutation skoruyla, deploy sıklığı başarısız değişiklik oranıyla birlikte okunur.

Kalitenin Tanımı ve İşlevleri

Kalite bir mesafedir

En soyut hâliyle kalite, bir sistemin yaptığı ile yapması gereken arasındaki mesafedir. Hata da bu mesafenin görünür hale geldiği tek bir noktadır. Bu tanımın üç parçası vardır:

1. **Niyet:** Sistemin ne yapması gerektiği. Çoğu zaman yazılı değildir, kafalarda durur.
2. **Davranış:** Sistemin gerçekte ne ürettiği.
3. **Karşılaştırma:** Niyet ile davranışı yan yana koyup uyup uymadığına karar veren mekanizma. Test dünyasındaki adı **test oracle**'dır.

Karşılaştırma yoksa sistem kördür

Bir hata yalnızca bir karşılaştırmanın yapıldığı yerde bulunur. Karşılaştırması olmayan bir niyet için sistem kördür; hata orada sessizce yaşar. Bu yüzden "neyi test edeyim?" sorusu, "hangi niyetin henüz bir karşılaştırması yok?" sorusuna dönüştürülebilir. İkinci soru daha kolay cevaplanır.

Karşılaştırmanın türleri

Hata bulma yolları, karşılaştırmanın neye karşı yapıldığına göre ayrılır:

Neye karşı	Örnek	Ne yakalar
Yazılı kurala	Unit test: "girdi X ise çıktı Y"	Bilinen beklentiden sapmayı
Kendine	Property-based testing: "iki kez sıralamak bir kez sıralamakla aynıdır"	Tek tek yazılamayan girdilerdeki hataları
Gerçeğe	Gerçek veride çalıştırmak, kullanıcı gözlemi	Hiç düşünülmemiş durumları
Geçmişe	Differential ve regression testing	Değişikliğin bozduğu şeyi
Zorlamaya	Fuzzing, chaos testing, yük testi	Sınırlardaki kırılmayı
İnsan okumasına	Kod incelemesi	Niyetin hiç yazılmadığı yerleri

Karşılaştırma ağının ölçüleri

Bir tespit sisteminin gücü üç ölçüye bağlıdır:

- **Genişlik:** Niyetlerin kaçının bir karşılaştırması var?
- **Derinlik:** Her karşılaştırma ne kadar çeşitli girdide çalışıyor? Tek bir örnekte mi, binlerce rastgele girdide mi, gerçek veride mi?
- **Hız:** Karşılaştırma ne zaman çalışıyor? Aynı hatayı kod yazılırken yakalamak ile canlı ortamda yakalamak arasında büyük maliyet farkı vardır.

Testin de kalitesi vardır

Karşılaştırmanın kendisi de yanılabilir. Hiç kırmızı yanmamış bir test, hatayı yakalayabildiğini kanıtlamamıştır. Kodu bilerek bozup testin başarısız olduğunu görmek, testin kendi işini yaptığını gösterir. Bu fikrin sistemli hâli **mutation testing**'dir. Araç koda küçük hatalar ekler ve testlerin bunların yüzde kaçını yakaladığını ölçer. Bu orana **mutation skoru** denir.

İki ayrı mesafe

Kalite tek bir mesafe değildir. Zincir şöyledir:

$$\text{ihtiyaç} \rightarrow \text{niyet} \rightarrow \text{davranış}$$

Bu zincirin iki halkasında ayrı ayrı kayma olabilir:

- **Verification** (doğrulama): Davranış niyete uyuyor mu? Yani "doğru yaptık mı?" Testlerin ölçtüğü mesafe budur.
- **Validation** (geçerleme): Niyet gerçek ihtiyaca uyuyor mu? Yani "doğru şeyi mi yaptık?" Kusursuz çalışan ama kimsenin işine yaramayan bir özellik de kalitesizdir. Bu mesafeyi ancak gerçek kullanım gösterir.

Zaman boyutu

Bugün iyi çalışan bir sistem, değiştirilmesi zorsa yarın kalitesizleşir. Çünkü ihtiyaç değişir ve sistem ona yetişemez. Bu yüzden kalitenin üçüncü parçası **değişime dayanıklılıktır**: mesafe açıldığında onu ne kadar kolay kapatabildiğin.

Test kaliteyi yaratmaz

Test kaliteyi ölçer, yaratmaz. Hatayı görmek ile hatanın hiç olmaması farklı şeylerdir. Örneğin kullanıcı kimliği ile dosya kimliği ayrı tipler olarak tanımlandığında, ikisini karıştırma hatası hiç yazılamaz. Veritabanında `NOT NULL` kısıtı varsa boş veri giremez. Bu durumlarda ortada ne test ne de karşılaştırma vardır; hatanın saklanabileceği yer ortadan kalkmıştır.

Kalite sürecinin dört işlevi

Kalite süreci hataya karşı dört ayrı iş yapar. Bunlar aşama değildir; sırayla bitip geçilmezler, hepsi aynı anda ve sürekli çalışır. Ancak her biri hatanın hayatında farklı bir ana denk gelir:

İşlev	Hatanın hangi anında	Sorduğu soru
Önleme	Hata doğmadan önce	Bu hata yazılabilir mi?
Tespit	Hata doğduktan sonra, kullanıcıya ulaşmadan	Bu hata var mı?
Sınırlama	Hata kullanıcıya ulaştığında	Ne kadar zarar verecek?
Öğrenme	Hata çözüldükten sonra	Bir daha olur mu?

Bu dörtlü yazılıma özgü değildir. Güvenlik mühendisliği, havacılık ve tıpta aynı ayrım kullanılır: *prevention*, *detection*, *mitigation*, *learning*. Tıptaki karşılığı şöyledir: hastalık aşıyla önlenir, taramayla erken teşhis edilir, tedaviyle zararı sınırlanır, vaka incelemesiyle sistem düzeltilir.

Önleme

Hatanın saklanabileceği yerleri azaltır. Araçları: TDD, küçük fonksiyonlar, düşük karmaşıklık, geçersiz durumu temsil edilemez yapan tipler, fail fast, assertion, veritabanı kısıtları.

Tespit

Var olan hatayı kullanıcıdan önce görünür kılar. Araçları: unit ve integration test, kod incelemesi, fuzzing, property-based testing, statik analiz, race detector.

Sınırlama

Hata yine olur, ama yalnızca küçük bir kısmı etkiler ve geri alınabilir. Kullanıcının gördüğü şey değiştiği için bu da kalitedir. Araçları: canary deploy, feature flag, rollback, yedek, hata takibi.

Öğrenme

Bir hatayı düzeltmekle kalmaz, aynı türden hatanın bir daha gelmemesini sağlar. Araçları: regression testi, postmortem, game day, hotspot analizi.

İşlev ve araç ayrımı

- **İşlev:** Kalitenin ne yaptığı.
- **Katman ya da teknik:** Bunu nasıl yaptığı.

Örneğin TDD önleme işlevine, fuzzing tespite, canary deploy sınırlamaya, postmortem öğrenmeye hizmet eder. İyi bir kalite sistemi dört işlevin hiçbirini boş bırakmaz. Yalnızca teste yatırım yapan bir ekip, tespiti güçlü ama önleme, sınırlama ve öğrenmesi zayıf bir sistem kurmuş olur.

Emeğin dağılımı

Dört işlevin altında ortak bir karar yatar: emeğin nereye harcanacağı. Hatalar kodda eşit dağılmaz; büyük kısmı modüllerin küçük bir bölümünde toplanır. Bu yüzden kalite, her yere aynı miktarda test yazmak değil, emeği riskin yoğun olduğu yere yöneltmektir.

Özet tanım

Kalite, niyet ile davranış arasındaki mesafeyi küçük tutma işidir. Tasarım bu mesafenin açılmasını önler. Test açıldığını gösterir. Canlı ortamdaki önlemler açıldığında zararı sınırlar. Öğrenme de aynı yerden bir daha açılmamasını sağlar. Test bu zincirin yalnızca ortadaki halkasıdır.

En kısa hâliyle: **kalite, doğru şeyi doğru yapmak ve öyle kalabilmektir.**

Katmanların Toplu Görünümü

Katman	Başlıca teknikler
Önleme	TDD, boyut ve karmaşıklık sınırları, guard clause, geçersiz durumu temsil edilemez yapmak, fail fast, assertion, veritabanı kısıtları, strict tipler, mimari testleri
İnceleme	200–400 satırlık PR, tek amaçlı PR, kontrol listesi, Fagan inspection, pair programming, gereksinim incelemesi, BDD, threat modeling, FMEA
Test tasarımı	Test piramidi, sınır değer, denklik sınıfları, karar tablosu, durum geçiş, pairwise, error guessing, golden test, characterization test, contract test, testcontainers, fake clock
Otomatik arama	Fuzzing, property-based, differential, metamorphic, mutation, symbolic execution, statik analiz, race detector, sanitizer
Sistem	Chaos testing, failpoint, ağ bölünmesi, saat kayması, yük/spike/soak, deterministic simulation, TLA+, N-version
Arayüz	Visual regression, cross-browser, pseudo-localization, erişilebilirlik, heap snapshot, monkey testing
Güvenlik	SAST, DAST, bağımlılık ve imaj taraması, penetrasyon testi, bug bounty
Canlı ortam	Quality gate, feature flag, canary, dark launch, shadow traffic, hata takibi, tracing, SLO, synthetic monitoring, checksum, mutabakat
Öğrenme	Regression testi, `git bisect`, delta debugging, postmortem, game day, hotspot analizi, defect prediction
Ölçüm	DORA metrikleri, coverage, mutation skoru, benchmark karşılaştırması

Sonuç

Kalite tek bir teknikle değil, katmanların birlikte çalışmasıyla elde edilir. Bu ilkenin pratik karşılığı, her katmanın en ucuz ve en etkili birkaç tekniğinin seçilmesidir. Bütün tekniklerin uygulanması ne mümkün ne de gereklidir. Çoğu proje için başlangıç noktası şu çekirdek kümedir: küçük PR'lar, zorunlu linter ve race detector, sınır değerleri kapsayan unit testler, kritik ayrıştırıcılar için fuzzing, her bug için regression testi ve canlıda hata takibi. Diğer teknikler, sistemin riskine göre eklenir.

Kalite çalışmasının en sık yapılan hatası emeğin eşit dağıtılmasıdır. Hatalar kodda eşit dağılmadığı için test emeği de eşit dağıtılmamalıdır. Hotspot analizi, hata geçmişi ve FMEA, emeğin nereye gideceğini belirleyen araçlardır.

İkinci sık hata, ölçüyü amaçla karıştırmaktır. Coverage oranı, satır sınırı ya da deploy sıklığı birer göstergedir. Tek başına hedef yapıldıklarında ekip göstergeyi iyileştirir ama kaliteyi iyileştirmez. Göstergeler birbirini dengeleyecek biçimde birlikte okunmalıdır.

Çerçevenin bir sınırlılığı vardır: tekniklerin çoğunun maliyeti ile kazancı projeden projeye çok değişir ve bunu önceden hesaplamak zordur. Hataların ne kadar erken bulunursa o kadar ucuz düzeltildiği yaygın olarak kabul edilir (Boehm, 1981); ancak bu iddiada sıkça alıntılanan kesin maliyet oranlarının dayanağı zayıftır ve eleştirilmiştir (Bossavit, 2015). Yönü doğru olan bu ilkenin büyüklüğü belirsizdir.

Açık kalan soru şudur: kalite yatırımı ne zaman yeterlidir? Hiçbir teknik bug'ın yokluğunu kanıtlayamadığı için bu sorunun teknik bir cevabı yoktur. Cevap, bir hatanın kullanıcıya ve işe maliyeti ile onu önlemenin maliyeti arasındaki dengeye göre her sistem için ayrıca verilir. Yayın sistemleri, ödeme sistemleri ve sağlık yazılımları bu dengede önlemeye çok daha fazla ağırlık verir.

Kaynaklar

- Avizienis, A. (1985). The N-Version Approach to Fault-Tolerant Software. *IEEE Transactions on Software Engineering*, 11(12).
- Beck, K. (2002). *Test-Driven Development: By Example*. Addison-Wesley.
- Beyer, B., Jones, C., Petoff, J., Murphy, N. R. (2016). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly.
- Boehm, B. W. (1981). *Software Engineering Economics*. Prentice Hall.
- Bossavit, L. (2015). *The Leprechauns of Software Engineering*. Leanpub.
- Chen, T. Y., Cheung, S. C., Yiu, S. M. (1998). Metamorphic Testing: A New Approach for Generating Next Test Cases. Technical Report HKUST-CS98-01, Hong Kong University of Science and Technology.
- Claessen, K., Hughes, J. (2000). QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. *Proceedings of ICFP 2000*.

- Cohen, D. M., Dalal, S. R., Fredman, M. L., Patton, G. C. (1997). The AETG System: An Approach to Testing Based on Combinatorial Design. *IEEE Transactions on Software Engineering*, 23(7).
- Cohen, J. (2006). *Best Kept Secrets of Peer Code Review*. Smart Bear.
- Cohn, M. (2009). *Succeeding with Agile*. Addison-Wesley.
- DeMillo, R. A., Lipton, R. J., Sayward, F. G. (1978). Hints on Test Data Selection: Help for the Practicing Programmer. *IEEE Computer*, 11(4).
- Dijkstra, E. W. (1970). *Notes on Structured Programming*. EWD249.
- Duvall, P. M., Matyas, S., Glover, A. (2007). *Continuous Integration: Improving Software Quality and Reducing Risk*. Addison-Wesley.
- Fagan, M. E. (1976). Design and Code Inspections to Reduce Errors in Program Development. *IBM Systems Journal*, 15(3).
- Feathers, M. (2004). *Working Effectively with Legacy Code*. Prentice Hall.
- Fenton, N. E., Neil, M. (1999). A Critique of Software Defect Prediction Models. *IEEE Transactions on Software Engineering*, 25(5).
- Fenton, N. E., Ohlsson, N. (2000). Quantitative Analysis of Faults and Failures in a Complex Software System. *IEEE Transactions on Software Engineering*, 26(8).
- Forsgren, N., Humble, J., Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps*. IT Revolution.
- Fowler, M. (1999). *Refactoring: Improving the Design of Existing Code*. Addison-Wesley.
- Godefroid, P., Klarlund, N., Sen, K. (2005). DART: Directed Automated Random Testing. *Proceedings of PLDI 2005*.
- Humble, J., Farley, D. (2010). *Continuous Delivery*. Addison-Wesley.
- Hunt, A., Thomas, D. (1999). *The Pragmatic Programmer*. Addison-Wesley.
- Inozemtseva, L., Holmes, R. (2014). Coverage Is Not Strongly Correlated with Test Suite Effectiveness. *Proceedings of ICSE 2014*.
- ISO/IEC (2011). *ISO/IEC 25010: Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models*.
- Ivanković, M., Petrović, G., Just, R., Fraser, G. (2019). Code Coverage at Google. *Proceedings of ESEC/FSE 2019*.

- King, J. C. (1976). Symbolic Execution and Program Testing. *Communications of the ACM*, 19(7).
- Knight, J. C., Leveson, N. G. (1986). An Experimental Evaluation of the Assumption of Independence in Multiversion Programming. *IEEE Transactions on Software Engineering*, 12(1).
- Kuhn, D. R., Wallace, D. R., Gallo, A. M. (2004). Software Fault Interactions and Implications for Software Testing. *IEEE Transactions on Software Engineering*, 30(6).
- Lamport, L. (2002). *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley.
- Luo, Q., Hariri, F., Eloussi, L., Marinov, D. (2014). An Empirical Analysis of Flaky Tests. *Proceedings of FSE 2014*.
- Martin, R. C. (2008). *Clean Code*. Prentice Hall.
- McCabe, T. J. (1976). A Complexity Measure. *IEEE Transactions on Software Engineering*, 2(4).
- McKeeman, W. M. (1998). Differential Testing for Software. *Digital Technical Journal*, 10(1).
- Meyer, B. (1992). Applying "Design by Contract". *IEEE Computer*, 25(10).
- Miller, B. P., Fredriksen, L., So, B. (1990). An Empirical Study of the Reliability of UNIX Utilities. *Communications of the ACM*, 33(12).
- Minsky, Y. (2011). Effective ML. Harvard Üniversitesi'nde verilen konuşma. Kaynağın yayın biçimi doğrulanmalıdır.
- Myers, G. J. (1979). *The Art of Software Testing*. Wiley.
- North, D. (2006). Introducing BDD. *Better Software Magazine*.
- Reason, J. (1990). *Human Error*. Cambridge University Press.
- Rosenthal, C., Jones, N. (2020). *Chaos Engineering: System Resiliency in Practice*. O'Reilly.
- Shostack, A. (2014). *Threat Modeling: Designing for Security*. Wiley.
- Tornhill, A. (2015). *Your Code as a Crime Scene*. Pragmatic Bookshelf.
- Utting, M., Legeard, B. (2007). *Practical Model-Based Testing: A Tools Approach*. Morgan Kaufmann.
- Zeller, A., Hildebrandt, R. (2002). Simplifying and Isolating Failure-Inducing Input. *IEEE Transactions on Software Engineering*, 28(2).

- Zhou, J. vd. (2021). FoundationDB: A Distributed Unbundled Transactional Key Value Store. *Proceedings of SIGMOD 2021*.